**MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA**

**Technical University of Moldova**
**Faculty of Computers, Informatics and Microelectronics**
**Department of Software and Automation Engineering**

**Oberst Eduard, FAF-243**

# Report

Laboratory work nr. 3

## The Command Line: Shell, Pipes and Text Filters

Course: Operating Systems (Track A: Linux)

*Checked by:*
**Patricia Reitman**, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

# Purpose of the work

The purpose of this laboratory work is to learn to work quickly in bash, to predict what the shell does to a command line before it runs (globbing, quoting, variable, arithmetic and command substitution), to redirect standard output and standard error where needed, to chain commands with `;`, `&&` and `||`, to control background jobs, and to answer real questions about data with one line of small text filters (`grep`, `cut`, `sort`, `uniq`, `wc`, `head`, `tail`, `tr`) joined by pipes. Finally, the best one-liners are turned into a small bash script.

All work was done on Ubuntu 26.04.1 LTS in bash. This Ubuntu release ships the Rust implementation of the core utilities (uutils), which is why the help text of `ls` looks slightly different from GNU coreutils.

### Preparation: the data set

```
~$ mkdir -p ~/os-lab3 && cd ~/os-lab3
~/os-lab3$ for i in $(seq 1 500); do
  echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $((i % 5)) -eq 0 ] \
&& echo FAIL || echo OK) /page$((i % 4))"
done > access.log
~/os-lab3$ head -3 access.log ; wc -l access.log
2026-10-05 10:01 user1 OK /page1
2026-10-05 10:02 user2 OK /page2
2026-10-05 10:03 user3 OK /page3
500 access.log
```

# Part 1. Move fast, find help

### Interesting Command Output:

```
~$ ls /usr/sh<Tab><Tab>
~$ ls /usr/share/
~$ cd /et<Tab>
~$ cd /etc/
```

Figure 1.1 – Tab completion

For `ls /usr/sh` there is only one match, so the first Tab already completes the word to `/usr/share/`; the second Tab lists nothing more because there is no ambiguity. For `cd /et` one Tab completes to `/etc/`. With several matches the first Tab would only beep and the second would list all candidates. After running three commands, Ctrl+R opened `(reverse-i-search)`; typing the first few letters of the first command found it in the history, and Enter executed it again.

```
~/os-lab3$ type cd ; type ls ; type echo ; which ls
cd is a shell builtin
ls is aliased to `ls --color=auto'
echo is a shell builtin
/usr/bin/ls
```

Figure 1.2 – Output of type and which

```
~/os-lab3$ echo $PATH | tr ':' '\n'
/home/capziro/.local/bin
/home/capziro/bin
/home/capziro/.nvm/versions/node/v24.20.0/bin
/home/capziro/.local/bin
/home/capziro/bin
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/usr/games
/usr/local/games
/snap/bin
/snap/bin
~/os-lab3$ echo $PATH | tr ':' '\n' | wc -l
15
```

Figure 1.3 – The directories in PATH

```
~/os-lab3$ nosuchprog ; echo "exit code: $?"
nosuchprog: command not found
exit code: 127
```

Figure 1.4 – Exit code of a missing command

```
~/os-lab3$ man -k "list directory"
dir (1)              - List directory contents. Ignore files and directories ...
gnuls (1)            - list directory contents
ls (1)               - List directory contents. Ignore files and directories ...
vdir (1)             - List directory contents. Ignore files and directories ...
~/os-lab3$ man ls          (inside: /sort, n)
       -t     Sort by modification time (the 'mtime' in the inode), newest first.
~/os-lab3$ help cd | head -3
cd: cd [-L|[-P [-e]]] [-@] [dir]
    Change the shell working directory.
```

Figure 1.5 – Finding help without a browser

**Observations:**

1. **Which of cd, ls, echo are built-ins, and why MUST cd be one?**

   `cd` and `echo` are shell built-ins; `ls` is an external program (`/usr/bin/ls`, in an interactive shell also wrapped by the alias `ls --color=auto`).

`cd` must be a built-in because the current working directory is a property of each process. An external `cd` would run in a child process, call `chdir()` there, and exit; the parent shell's directory would stay unchanged. Only the shell itself can change its own directory.

2. **How many directories are in your PATH, and in which one is ls?**

   PATH contains 15 entries (12 distinct directories, because `~/.local/bin`, `~/bin` and `/snap/bin` are added twice by the startup files). `ls` is found in `/usr/bin` (`/bin` is a symlink to `/usr/bin`).

3. **What exit code does a command that is not found return?**

   127 – bash prints `command not found` and sets `$?` to 127.

4. **Which option of ls sorts by modification time?**

   `-t` (sort by modification time, newest first); found in `man ls` with `/sort` and `n`.

# Part 2. What the shell does to your line

### Interesting Command Output:

```
~$ mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
~/os-lab3/play$ touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
~/os-lab3/play$ ls
 a.c  'alfa beta.txt'   b.c   file1.txt   file2.txt   notes.txt
```

Figure 2.1 – Files created in the play directory

```
~/os-lab3/play$ echo *.c
a.c b.c
~/os-lab3/play$ echo file?.txt
file1.txt file2.txt
~/os-lab3/play$ echo [a-b]*
a.c alfa beta.txt b.c
~/os-lab3/play$ echo *.xyz
*.xyz
~/os-lab3/play$ echo {1..5} ; echo $((7 * 6))
1 2 3 4 5
42
~/os-lab3/play$ echo "today is $(date +%A), I am $(whoami)"
today is Monday, I am capziro
~/os-lab3/play$ touch log{1..3}.txt && ls log*
log1.txt  log2.txt  log3.txt
```

Figure 2.2 – Globbing results

```
~/os-lab3/play$ ls alfa beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
~/os-lab3/play$ ls "alfa beta.txt"
'alfa beta.txt'
~/os-lab3/play$ ls alfa\ beta.txt
'alfa beta.txt'
```

Figure 2.3 – The ls error on a name with a space, and the fix

```
~/os-lab3/play$ echo '$HOME' "$HOME" \$HOME
$HOME /home/capziro $HOME
```

Figure 2.4 – Single quotes, double quotes and backslash

Before running each line I wrote down a prediction. The results are summarised in Table 2.1.

Table 2.1 – Predictions versus real output

| **Command** | **My prediction** | **Real output** | **Explanation (if different)** |
|---|---|---|---|
| `echo *.c` | a.c b.c | a.c b.c | – |
| `echo file?.txt` | file1.txt file2.txt | file1.txt file2.txt | – |
| `echo [a-b]*` | a.c b.c | a.c alfa beta.txt b.c | `[a-b]*` means "first letter a or b, then anything", so `alfa beta.txt` also matches; I forgot it starts with a. |
| `echo *.xyz` | (empty line) | *.xyz | When a pattern matches nothing, bash leaves the word unchanged, so echo prints it literally. |
| `ls alfa beta.txt` | error: no such file | two errors: 'alfa' and 'beta.txt' | Word splitting gives ls two arguments, so ls reports each one separately. |
| `ls "alfa beta.txt"` | alfa beta.txt | 'alfa beta.txt' | Works; ls shows the name in quotes on a terminal (see the note below). |
| `echo '$HOME' "$HOME" \ $HOME` | $HOME /home/capziro $HOME | $HOME /home/capziro $HOME | – |
| `echo {1..5} ; echo $((7 * 6))` | 1 2 3 4 5 / 42 | 1 2 3 4 5 / 42 | – |
| `echo "today is $(date +%A), I am $(whoami)"` | today is Monday, I am capziro | today is Monday, I am capziro | – |
| `touch log{1..3}.txt && ls log*` | log1.txt log2.txt log3.txt | log1.txt log2.txt log3.txt | – |

**Observations:**

1. **For each line: your prediction, the real output, and one sentence if they differ.**

   See Table 2.1. Two predictions were wrong: `echo [a-b]*` also matched `alfa beta.txt`, and `echo *.xyz` printed the pattern itself instead of an empty line.

2. **echo knows nothing about files, yet echo *.c printed file names. Who produced them?**

   The shell. Before starting `echo`, bash performs pathname expansion (globbing): it replaces `*.c` with the sorted list of matching file names. `echo` just receives the arguments `a.c` and `b.c` and prints them.

3. **Why did ls alfa beta.txt fail? Give two different ways to fix it.**

   After expansion the shell splits the line into words at unquoted spaces, so `ls` received two arguments, `alfa` and `beta.txt`, and neither file exists. Fixes:

   - quote the name: `ls "alfa beta.txt"` or `ls 'alfa beta.txt'`;
   - escape the space with a backslash: `ls alfa\ beta.txt` (Tab completion inserts it automatically).

4. **What is the difference between single and double quotes?**

   Inside single quotes nothing is special: `'$HOME'` stays the literal text `$HOME`. Inside double quotes the text is kept as one word (no word splitting, no globbing), but `$VAR`, `$(command)` and `$((arithmetic))` are still expanded, so `"$HOME"` becomes `/home/capziro`. A backslash quotes only the next character (`\$HOME` prints `$HOME`).

   **Note.** On a terminal `ls` prints the file as `'alfa beta.txt'` in quotes. Modern `ls` uses the "shell-escape" quoting style for names that contain spaces or special characters, so the name shown can be copied and pasted back into a command safely and is not mistaken for two files. When the output goes to a pipe or file (or with `ls -N`) the quotes are not added.

# Part 3. Redirection, chaining and jobs

### Interesting Command Output:

```
~/os-lab3$ ls /etc /nope > out.txt 2> err.txt
~/os-lab3$ wc -l out.txt ; cat err.txt
248 out.txt
ls: cannot access '/nope': No such file or directory
~/os-lab3$ ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
xml
zsh_command_not_found
~/os-lab3$ echo first > notes.md ; echo second >> notes.md ; cat notes.md
first
second
~/os-lab3$ wc -l < access.log
500
~/os-lab3$ wc -l access.log
500 access.log
```

Figure 3.1 – stdout and stderr in separate files

```
~/os-lab3$ ls /etc /nope > all.txt 2>&1
~/os-lab3$ wc -l < all.txt ; grep -c nope all.txt
249
1
~/os-lab3$ ls /etc /nope 2>&1 > all.txt
ls: cannot access '/nope': No such file or directory
~/os-lab3$ wc -l < all.txt ; grep -c nope all.txt
248
0
```

Figure 3.2 – Comparing the order of redirections

```
~/os-lab3$ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2
~/os-lab3$ mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
I am in /home/capziro/os-lab3/tmp
```

Figure 3.3 – The ; && || operators

```
~/os-lab3$ sleep 100 &
[1] 37812
~/os-lab3$ jobs ; kill %1 ; jobs
[1]+  Running                 sleep 100 &
[1]+  Terminated              sleep 100
```

Figure 3.4 – Running a background job and stopping it

### Observations:

1. **Which file got the error message, and which got the listing?**

   `err.txt` (file descriptor 2, stderr) got the error `ls: cannot access '/nope'`; `out.txt` (file descriptor 1, stdout) got the 248-line listing of `/etc`. Nothing

appeared on the screen. With `> all.txt 2>&1` both went into `all.txt` (249 lines); this `ls` writes the error first, so it is the first line of `all.txt` and `tail -2` shows only names from `/etc`.

2. **What is the difference between > all.txt 2>&1 and 2>&1 > all.txt?**

   The shell applies redirections from left to right, and `2>&1` means "make fd 2 a copy of what fd 1 points to right now".

   - `> all.txt 2>&1`: first stdout is pointed to `all.txt`, then stderr is copied from stdout, so both go to the file (249 lines, the error included).
   - `2>&1 > all.txt`: first stderr is copied from stdout, which is still the terminal; only then is stdout moved to the file. The error is printed on the screen and `all.txt` has only the 248 names.

3. **Why does wc -l < access.log print only a number, while wc -l access.log also prints the file name?**

   With `<` the shell opens the file and connects it to the standard input of `wc`; `wc` reads stdin and never learns any file name, so it prints only `500`. With an argument, `wc` opens the file itself, knows its name and prints `500 access.log`.

4. **Explain each operator in one sentence: ; && ||**

   - `;` runs the next command after the previous one finishes, whatever its exit code.
   - `&&` runs the next command only if the previous one succeeded (exit code 0).
   - `||` runs the next command only if the previous one failed (non-zero exit code), e.g. `ls /nope 2> /dev/null || echo "failed with code $?"` printed `failed with code 2`.

# Part 4. Answer questions with one-liners

Each line of `access.log` is: date, time, user, result (OK or FAIL), page. Every question is answered with one command line; all lines are saved in `oneliners.sh`, with the question as a comment above.

**Interesting Command Output:**

```
~/os-lab3$ grep -cw FAIL access.log
100
```

Figure 4.1 – Number of failed requests

```
~/os-lab3$ cut -d' ' -f5 access.log | sort -u | cat -n
     1  /page0
     2  /page1
     3  /page2
     4  /page3
```

Figure 4.2 – Number and names of the different pages

```
~/os-lab3$ cut -d' ' -f5 access.log | sort | uniq -c
    125 /page0
    125 /page1
    125 /page2
    125 /page3
```

Figure 4.3 – Requests per page

```
~/os-lab3$ grep -w FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn
     15 user5
     15 user3
     14 user6
     14 user4
     14 user2
     14 user1
     14 user0
~/os-lab3$ grep -w FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn \
           | awk 'NR==1 {max=$1} $1==max'
     15 user5
     15 user3
```

Figure 4.4 – Users with the most failed requests

```
~/os-lab3$ echo "user3: $(grep -cw user3 access.log) requests, \
$(grep -w user3 access.log | grep -cw FAIL) failed"
user3: 72 requests, 15 failed
```

Figure 4.5 – Requests and failures of user3

```
~/os-lab3$ grep -w FAIL access.log | tail -3 | cut -d' ' -f2,3
10:10 user0
10:15 user5
10:20 user3
```

Figure 4.6 – The last 3 failed requests (time and user)

```
~/os-lab3$ cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
     41 /usr/sbin/nologin
      5 /bin/false
      2 /bin/bash
      1 /bin/sync
```

Figure 4.7 – Login shells in /etc/passwd

```
~/os-lab3$ ls /etc | wc -l
247
~/os-lab3$ ls -l /etc | wc -l
248
~/os-lab3$ ls -l /etc | head -1
total 1224
```

Figure 4.8 – ls versus ls -l line count

**Check.** The four counts of Q3 add up to $4 \times 125 = 500$ (the number of lines in the log), and the per-user failure counts behind Q4 add up to $15 + 15 + 5 \times 14 = 100$, which equals the answer of Q1.

**Observations:**

1. **Q1. How many requests failed?**

   100 requests failed (every fifth line). `grep -w` matches FAIL only as a whole word and `-c` counts the matching lines.

2. **Q2. How many different pages were requested, and which?**

   4 different pages: `/page0`, `/page1`, `/page2`, `/page3`. `cut` takes field 5, `sort -u` keeps one copy of each, and `cat -n` numbers them, so the last number is the count.

3. **Q3. How many requests did each page get?**

   Each page got exactly 125 requests (`uniq -c` counts equal adjacent lines, which is why `sort` comes first).

4. **Q4. Which user or users have the most failed requests, and how many?**

   Two users are tied: `user3` and `user5`, with 15 failed requests each; all others have 14. Because of the tie, `head -1` would be wrong, so the last filter prints every line equal to the maximum.

5. **Q5. How many requests did user3 make, and how many of them failed?**

   user3 made 72 requests, 15 of which failed.

6. **Q6. Print the last 3 failed requests, showing only time and user.**

   `10:10 user0`, `10:15 user5`, `10:20 user3`.

7. **Q7. Which login shells appear in /etc/passwd, and how many accounts use each?**

   `/usr/sbin/nologin` – 41 accounts, `/bin/false` – 5, `/bin/bash` – 2, `/bin/sync` – 1. Most accounts are system/service accounts that are not allowed to log in.

8. **Q8. PREDICT, then explain: why do ls /etc | wc -l and ls -l /etc | wc -l differ by exactly one?**

   Prediction: `ls -l` has one extra header line. Result: 247 versus 248. When output goes to a pipe `ls` prints one name per line, and `ls -l` also prints one line per entry, but it starts with an extra summary line `total 1224` (the disk blocks used by the listed files). That line is the difference.

**Full run of oneliners.sh**

```
~/os-lab3$ bash oneliners.sh | head -27
100
     1  /page0
     2  /page1
     3  /page2
     4  /page3
    125 /page0
    125 /page1
    125 /page2
    125 /page3
     15 user5
     15 user3
user3: 72 requests, 15 failed
10:10 user0
10:15 user5
10:20 user3
     41 /usr/sbin/nologin
      5 /bin/false
      2 /bin/bash
      1 /bin/sync
ls: 247  ls -l: 248  extra line: total 1224
2026-10-05 10:10 user3 FAIL /page2
2026-10-05 10:20 user6 FAIL /page0
...
50
```

Figure 4.9 – Full run of oneliners.sh

### The file oneliners.sh

```
# Q1. How many requests failed?
grep -cw FAIL access.log


# Q2. How many different pages were requested, and which?
cut -d' ' -f5 access.log | sort -u | cat -n


# Q3. How many requests did each page get?
cut -d' ' -f5 access.log | sort | uniq -c


# Q4. Which user or users have the most failed requests, and how many?
grep -w FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk 'NR==1 {max=$1} $1==max'


# Q5. How many requests did user3 make, and how many of them failed?
echo "user3: $(grep -cw user3 access.log) requests, $(grep -w user3 access.log | grep -cw FAIL) failed"


# Q6. Print the last 3 failed requests, showing only time and user.
grep -w FAIL access.log | tail -3 | cut -d' ' -f2,3


# Q7. Which login shells appear in /etc/passwd, and how many accounts use each?
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn


# Q8. Why do ls /etc | wc -l and ls -l /etc | wc -l differ by exactly one?
echo "ls: $(ls /etc | wc -l)  ls -l: $(ls -l /etc | wc -l)  extra line: $(ls -l /etc | head -1)"


# Bonus. Failed requests whose minute ends in 0 (10:00, 10:10, ...), and how many
grep -E ' 10:[0-5]0 user[0-9]+ FAIL ' access.log ; grep -cE ' 10:[0-5]0 user[0-9]+ FAIL ' access.log
```

Figure 4.10 – Contents of oneliners.sh

### Bonus: failed requests at a minute ending in 0

```
~/os-lab3$ grep -E ' 10:[0-5]0 user[0-9]+ FAIL ' access.log | head -3
2026-10-05 10:10 user3 FAIL /page2
2026-10-05 10:20 user6 FAIL /page0
2026-10-05 10:30 user2 FAIL /page2
~/os-lab3$ grep -cE ' 10:[0-5]0 user[0-9]+ FAIL ' access.log
50
```

Figure 4.11 – Bonus: grep -E for minutes ending in 0

There are **50** such lines. Every line with a minute ending in 0 has `i` divisible by 10, therefore also by 5, so all 50 of those lines are failures.

# Part 5. Your first script

### Interesting Command Output:

```
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
  echo "no such file: $log" >&2
  exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
for n in 0 1 2 3 4 5 6; do
  echo "user$n: $(grep -w "user$n" "$log" | grep -cw FAIL) failures"
done
```

Figure 5.1 – Contents of report.sh

```
~/os-lab3$ ./report.sh access.log
bash: ./report.sh: Permission denied
~/os-lab3$ chmod +x report.sh
~/os-lab3$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Figure 5.2 – Running the script on access.log

```
~/os-lab3$ ./report.sh nope.log ; echo "exit code: $?"
no such file: nope.log
exit code: 1
~/os-lab3$ report.sh access.log
report.sh: command not found
```

Figure 5.3 – Error handling for a missing file

The TODO is solved with a `for` loop over the numbers 0 to 6; for each user the lines of that user are selected with `grep -w "user$n"` and the FAIL lines among them are counted with `grep -cw FAIL`. The per-user numbers add up to 100, the same as the total.

**Observations:**

1. **What does the first line #!/bin/bash do, and what happens if you run ./report.sh before chmod +x?**

   `#!/bin/bash` is the shebang: when the file is executed, the kernel reads the first line and starts `/bin/bash` with the script as its argument. Before `chmod +x` the file has no execute permission, so the kernel refuses to run it: `Permission denied` (exit code 126). It could still be run as `bash report.sh`.

2. **Why must you type ./report.sh and not just report.sh?**

   For a name without a slash, the shell searches only the directories listed in PATH, and the current directory is not in PATH (for security reasons). `report.sh` therefore gives `command not found`, while `./report.sh` is an explicit path to the file in the current directory.

3. **Why does the script write the error with >&2 and end with exit 1?**

   `>&2` sends the message to standard error, so it reaches the user even when the normal output is redirected to a file or a pipe, and it does not mix with the real results. `exit 1` returns a non-zero exit code, so the caller (`$?`, `&&`, `||`, other scripts, cron) can detect that the script failed.

# Conclusion

In this laboratory work I learned to use bash efficiently and, more importantly, to understand what the shell does to a line before any program sees it. Tab completion, history search with Ctrl+R, `type`, `which`, `man` and `help` made it possible to move fast and find information without a browser. The prediction exercises showed that globbing, brace expansion, variable and command substitution, quote removal and word splitting are all done by the shell: `echo` and `ls` only receive the final list of arguments, which explains why `echo *.c` prints file names and why a file name with a space must be quoted.

Working with redirections I saw that standard output and standard error are separate streams that can be sent to different files, that the order of `>` and `2>&1` matters because redirections are applied from left to right, and that the operators `;`, `&&` and `||` use the exit code to control the flow of commands. Jobs started with `&` can be listed and stopped with `jobs` and `kill %1`.

Finally, by combining small filters (`grep`, `cut`, `sort`, `uniq -c`, `wc`, `tail`) with pipes, I answered every question about the access log with a single line and verified the results with independent checks (the page counts add up to 500 and the per-user failures to 100). Turning the best one-liners into `report.sh` showed the basics of shell scripting: the shebang, execute permission, arguments, error messages on stderr and meaningful exit codes.